

Final Report Project ISYS6197003 - Business Application Development



GROUP 2 LIBRARY MANAGEMENT SYSTEM LA11

2602050715 - JODY CHRISTIAN

2602054013 - NELSON THEO WILBERT

2602057040 - TOBIAS BENAYA

2602058075 - JOVAN ADELARD WIDODO

2602060426 - VANNESS JACKSON

2602061845 - ERICA NOVITA CHANDRA

2602066833 - WARREN PRESTON HARLIE

2602068656 - FELICIA CAHYA SUSANTO

2602069993 - CLAIRINE

2602072256 - BRANDON AXEL ANGKASA

2602074671 - GALEN NAYAKA NAYOTTAMA

2602081802 - RAFAEL OSVALDO WIJAYA

2602083846 - MARVELLA PANGESTU

2602216310 - EDMUNDUS WISNU PRASETYAJI

Chapter I - Preliminary

Background and objective

Background

Perpustakaan

Perpustakaan adalah pusat pengetahuan yang menyediakan akses kepada berbagai bahan bacaan dan sumber ilmu. Namun dalam era digital ini, tantangan untuk mempertahankan relevansi perpustakaan sebagai lembaga penyedia informasi semakin besar, perpustakaan menghadapi masalah dan kompleksitas, diantaranya:

- Manajemen koleksi yang besar
- Peminjaman yang memerlukan efisiensi tinggi
- Kebutuhan akses informasi yang cepat dan akurat bagi pengguna

Library Management System

Sistem manajemen perpustakaan (library management system) merupakan sebuah perangkat lunak / aplikasi yang dirancang untuk membantu dalam pengelolaan dan organisasi berbagai kegiatan di perpustakaan. LMS membantu otomatisasi berbagai tugas dan proses yang terlibat dalam pengelolaan koleksi perpustakaan, peminjaman, pengembalian dan kegiatan administratif lainnya.

LMS bukan hanya sekedar alat administratif, tetapi sebuah solusi yang mengintegrasikan teknologi untuk meningkatkan efisiensi dan kualitas layanan perpustakaan.

Objective

- Meningkatkan efisiensi dalam pengelolaan perpustakaan
- Memberikan akses cepat dan akurat kepada pengguna
- Meningkatkan pengalaman pengguna dalam pencarian

Roles and responsibility

- Team Leader
 - Jody Christian
- Design
 - Erica Novita C.
 - Clairine
 - Rafael Osvaldo W.
 - Marvella Pangestu
- Programming

- Vannes Jackson
- Jody Christian
- Jovan Adelard W.
- Galen Nayaka N.
- Testing
 - Nelson Theo W.
 - Felicia Cahya S.
 - Edmundus Wisnu P.
- Analisa & Presentasi
 - Brandon Axel A.
 - Warren Preston H.
 - Tobias Benaya

Timeline

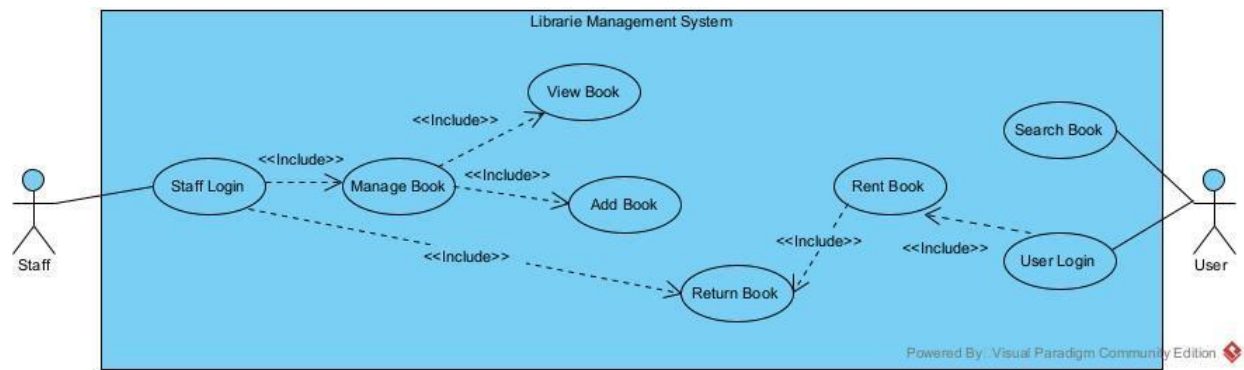
- 12-17 September 2023: Menentukan aplikasi yang ingin dibuat.
- 19-25 September 2023: Pembuatan Use Case.
- 26 September - 2 Oktober 2023: Mockup dan ERD.
- 3 - 9 Oktober 2023: Sprint #1.
- 10 - 16 Oktober 2023: Sprint #2 dan pembuatan Activity Diagram.
- 17 Oktober - 20 November 2023: Sprint #3.
- 21 November - 27 November: Sprint #4 dan Final Testing.
- 5 Desember 2023: Presentasi aplikasi Library Management System.
- 5-18 Desember 2023: Perbaikan dari feedback presentasi dan pembuatan Final Report.
- 19 Desember 2023: Pengumpulan final report.

Realisasi

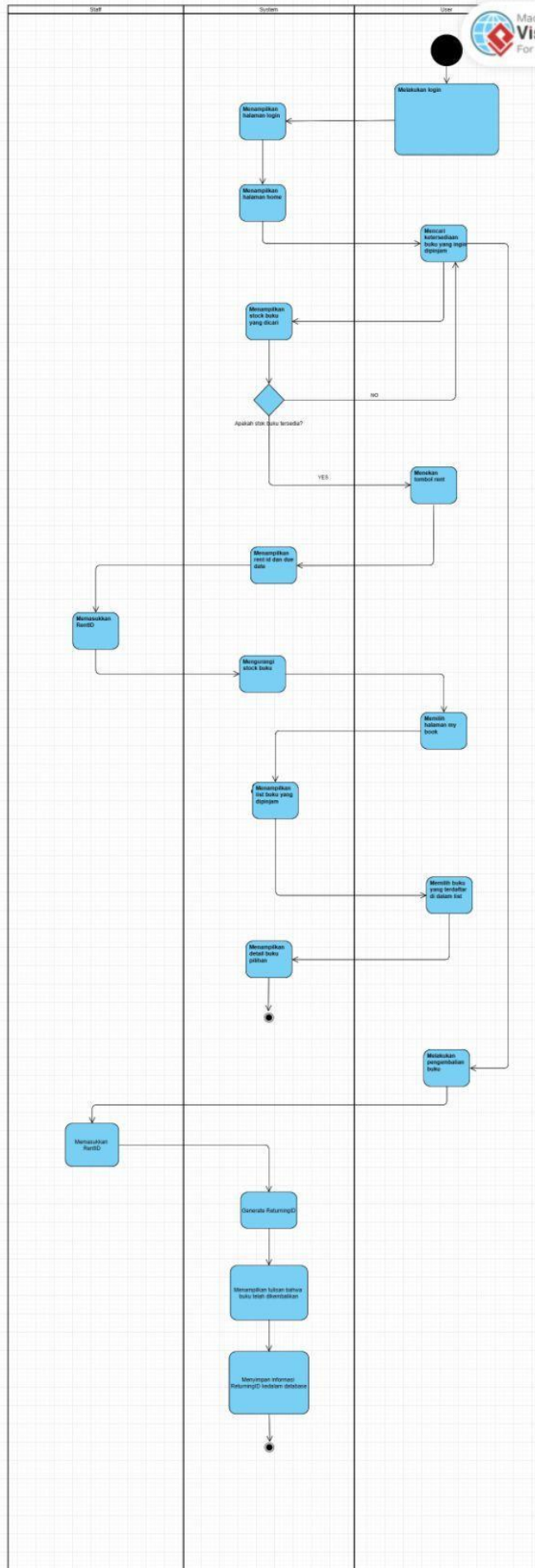
- 12-17 September 2023: Menentukan aplikasi Library Management System.
- 18 - 20 September 2023: Pembuatan Use Case.
- 21- 29 September 2023: Pembuatan Mockup.
- 30 September - 8 Oktober 2023: Pembuatan ERD.
- 9 - 16 Oktober 2023: Pembuatan Activity Diagram.
- 17- 21 Oktober 2023: Pembuatan Coding Sprint#1 dan menguji codingan.
- 22 - 23 Oktober 2023: Pembuatan Coding Sprint#2 dan menguji codingan.
- 24 Oktober 2023 - 19 November 2023: Pembuatan Coding Sprint#3 dan menguji codingan.
- 20 November 2023 - 28 November 2023: Pembuatan Coding Sprint#4 dan menguji codingan.
- 5-18 Desember 2023: Revisi program/coding, UI, SQL Query, dan Use Case dari hasil feedback.
- 19 Desember 2023: Pengumpulan final report.

Chapter II - Business Application

Use Case and Activity Diagram



Pada use case dari LMS yang kami kembangkan , terdapat dua aktor yaitu user dan staff. User, bisa melakukan pencarian buku dan melakukan user login. Setelah user berhasil melakukan user login, user juga bisa melakukan penyewaan buku dan include dengan pengembalian buku apabila user melakukan penyewaan buku. Pada sisi staff, dapat melakukan staff login. Setelah staff melakukan login, staff dapat melihat buku, menambah buku, dan mengurus pengembalian buku.



Diatas adalah Activity Diagram dari LMS yang kami kembangkan yang terdapat 3 actor, yaitu Staff, System dan juga User. activity diagram ini awalnya dimulai oleh User ketika User ingin log in ke akunnya pada aplikasi LMS, kemudian system akan menampilkan halaman log in dan ketika berhasil, system akan menampilkan halaman home. Lalu User mencari buku yang ingin dipinjam dan user dapat mengembalikan buku. System menampilkan ketersediaan buku yang dicari dengan adanya decision, terbuat 2 kemungkinan yaitu buku yang tidak tersedia dan membawa User untuk mencari buku yang lain atau buku tersedia dan pengguna bisa memilih tombol rent untuk meminjam buku. Lalu system akan menampilkan rentID dan juga due date untuk pengembalian buku, kemudian staff akan memasukkan rentID tersebut. System akan mengurangi data stock buku. lalu User dapat memilih my book, system akan menampilkan halaman buku yang dipinjam lalu User akan memilih buku untuk memilih buku yang terdaftar dan system akan menampilkan halaman detail buku pilihan.

User akan mengembalikan buku, staff akan memasukkan rentID dan system akan *generate* returningID buku tersebut, lalu menampilkan sebuah informasi bahwa buku tersebut telah dikembalikan dan system akan menyimpan returningID tersebut kedalam database.

GUI Design

Link Figma: <https://www.figma.com/file/5PeQVoO3YnuqniZx2pzKtg/Mockup-Librarie?type=design&node-id=17%3A752&mode=design&t=j8jH0IkTLnS8tPVc-1>

1. Halaman On Boarding

Ketika membuka aplikasi Librarie, maka aplikasi akan menampilkan halaman on-boarding



Keep Reading, Keep Exploring



2. Halaman Login

Apabila pengguna yang login adalah user, maka akan dialihkan ke halaman Home untuk user (nomor 2). Apabila pengguna yang login adalah staff, maka akan dialihkan ke halaman Home untuk admin (nomor 9)

3. Halaman Sign Up

Unlock the
world of knowledge.



Welcome to Librarie.

Unlock the world of knowledge.

Email

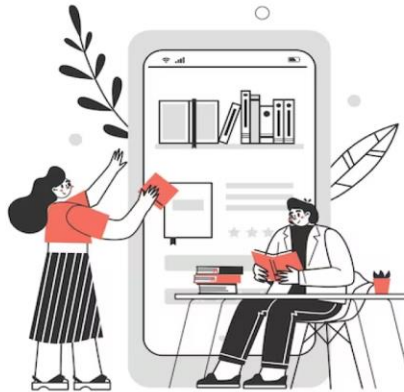
Password

[Forgot password?](#)

Login

Don't have an account? [Sign Up](#)

Unlock the
world of knowledge.



Sign up to Librarie

Name

Username

Email

Password

6+ characters

Sign Up

Already have an account? [Sign In](#)

4. Halaman Home

HOME My Books

Search more books or writers

Just For You

The Girl Who Fell Beneath The Sea
Axie Oh

Laut Ber cerita
Lella S. Chudori

Home Sweet Loan
Almira Bastari

Melangkah
J.S. Khairin

Funiculi Funicula 2
Toshikazu Kawaguchi

FILOSOFI TERAS
Henry Manampiring

[See more](#)

Trending Books

Book Lovers
Emily Henry

Meet Me Under the Mistletoe
Jenny Bayliss

Ghosting Writer
Aya Widjaja

Cantik Itu Luka
Eka Kurniawan

Good Vibes, Good Life
Vex King

Milk and Honey
Rupi Kaur

[See more](#)

5. Halaman Book Details

Ketika pengguna melakukan klik kepada salah satu buku, maka akan menampilkan halaman detail buku yang berisi status, jumlah buku, dan deskripsi. Pengguna bisa melike buku tersebut maupun menyimpan buku ke archive.



Book Lovers

Author: Emily Henry

Ratings:

Overview

Book Details

Availability Status : Available

Available to Rent : 15 copies

Like

Save

Share

Rent Now

Setelah menekan tombol “Rent Now”, maka aplikasi akan menampilkan kode RentID yang digunakan untuk ditunjukkan kepada staff perpustakaan ketika mengambil buku, dan menampilkan due rent buku.



You have rented the book:



Book Lovers

Author: Emily Henry

Ratings:

Please show the rent ID to the front office to pick up the book

Rent ID: NVL-00457

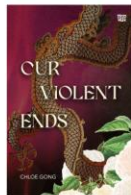
The book rent will due on:

28 October 2023

6. Halaman My Books

Halaman ini berisi buku-buku yang pernah dipinjam oleh pengguna (Rent History) dan buku yang sedang dipinjam oleh pengguna.

Rent History



Our Violent Ends
Chloe Gong



These Violent Delights
Chloe Gong



Eleanor & Park
Rainbow Rowell

Renting



Dallergut: Toko Penjual Mimpi
Lee Miye



Percy Jackson #1:
The Lightning Thief
Rick Riordan



Percy Jackson #2:
The Sea Of Monsters
Rick Riordan



Percy Jackson #3:
The Titans Curse
Rick Riordan

7. Halaman Archive

Halaman archive berisi list buku yang ingin disimpan oleh pengguna maupun list buku yang diberikan like oleh pengguna.

Saved Books



It Ends With Us
Colleen Hoover



The Roommate Pact
Allison Ashley



Hujan
Tere Liye

Liked Books



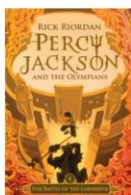
Pulang
Tere Liye



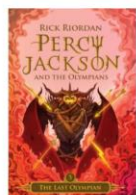
Sagaras
Tere Liye



Bumi
Tere Liye



Percy Jackson #4:
The Battle Of The Labyrinth
Rick Riordan



Percy Jackson #5:
The Last Olympian
Rick Riordan



Namaku Alam
Leila S. Chudori

8. Halaman Profile

 HOME My Books  



 **Jessica / General**
Update your username and manage your account

General
Edit Profile
Password
[Delete Account](#)

Username

Your library URL: <https://library//jessica>
Email

[Save Changes](#)

9. Halaman Home (staff)

 HOME Add Books Borrowing



Books



The Girl Who Fell Beneath The Sea
Axie Oh



Laut Bercerita
Lella S. Chudori



Home Sweet Loan
Almira Bastari



Melangkah
J.S. Khalren



Funiculi Funicula 2
Toshikazu Kawaguchi



Filosofi Teras
Henry Manampiring



Book Lovers
Emily Henry



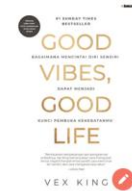
Meet Me Under the Mistletoe
Jenny Bayliss



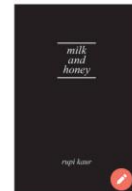
Ghosting Writer
Aya Widjaja



Cantik Itu Luka
Eka Kurniawan



Good Vibes, Good Life
Vex King



Milk and Honey
Rupi Kaur









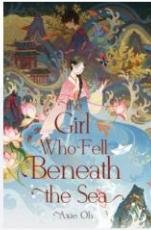




10. Halaman Detail Buku

 **HOME** Add Books Borrowing





The Girl Who Fell Beneath The Sea

Author: Axie Oh
Ratings:



Overview

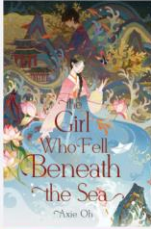
Badai mematikan telah merusak tanah kelahiran Mina selama beberapa generasi. Banjir menyapu seluruh desa, sementara perang berdarah dikobarkan untuk memperebutkan sumber daya yang tersisa. Masyarakat di desa Mina memercayai bahwa Dewa Laut mengutuk mereka dengan kematian dan keputusan. Dalam upaya untuk menenangkan Dewa Laut, setiap tahun seorang gadis cantik dibuang ke laut untuk menjadi pengantin Dewa Laut, dengan harapan suatu hari "pengantin sejati" akan dipilih dan mengakhiri penderitaan mereka. Shim Cheong adalah gadis tercantik di desa, sekaligus kekasih Joon. Banyak yang percaya dialah pengantin sejati Dewa Laut Tapi pada malam Cheong hendak dikorbankan, Joon mengikuti Cheong, meski tahu

11. Halaman Edit Book

Staff bisa melakukan edit buku dengan menekan ikon di halaman detail buku.

 **HOME** **Add Books** Borrowing

Edit Book



Title

The Girl Who Fell Beneath The Sea

Authors

Axie Oh

Overview

Badai mematikan telah merusak tanah kelahiran Mina selama beberapa generasi. Banjir menyapu seluruh desa, sementara perang berdarah dikobarkan untuk memperebutkan sumber daya yang tersisa. Masyarakat...

Upload New Cover

Save Changes

12. Halaman Add Book

Staff bisa menambahkan buku dengan mengakses navigasi “Add Books”

HOME Add Books Borrowing

Add Book

Title

Authors

Overview

Upload book cover

Delete

Add Book

13. Halaman Borrowing

Ketika pengguna mendatangi staff untuk meminjam buku, maka staff akan mengakses navigasi Borrowing untuk mengakses halaman Borrowing. Staff kemudian memasukkan rent ID (dari halaman pada nomor 3).

HOME Add Books Borrowing

Borrowing

Input Rent ID

Show Book

Setelah melakukan klik pada tombol “Show Book”, staff bisa melihat buku apa yang dipinjam oleh pengguna. Setelah mengambilkan buku dan memberikan kepada pengguna,

maka staff bisa mengklik pada “Book Rent” untuk menandakan bahwa buku yang ingin dipinjam telah diambil oleh pengguna. Ketika pengguna ingin mengembalikan buku, pengguna kembali memberikan rent ID untuk memunculkan informasi mengenai buku. Apabila buku sudah dikembalikan, maka staff akan mengklik tombol “Book Return” untuk menandakan bahwa buku yang dipinjam telah dikembalikan.

 HOME Add Books **Borrowing**



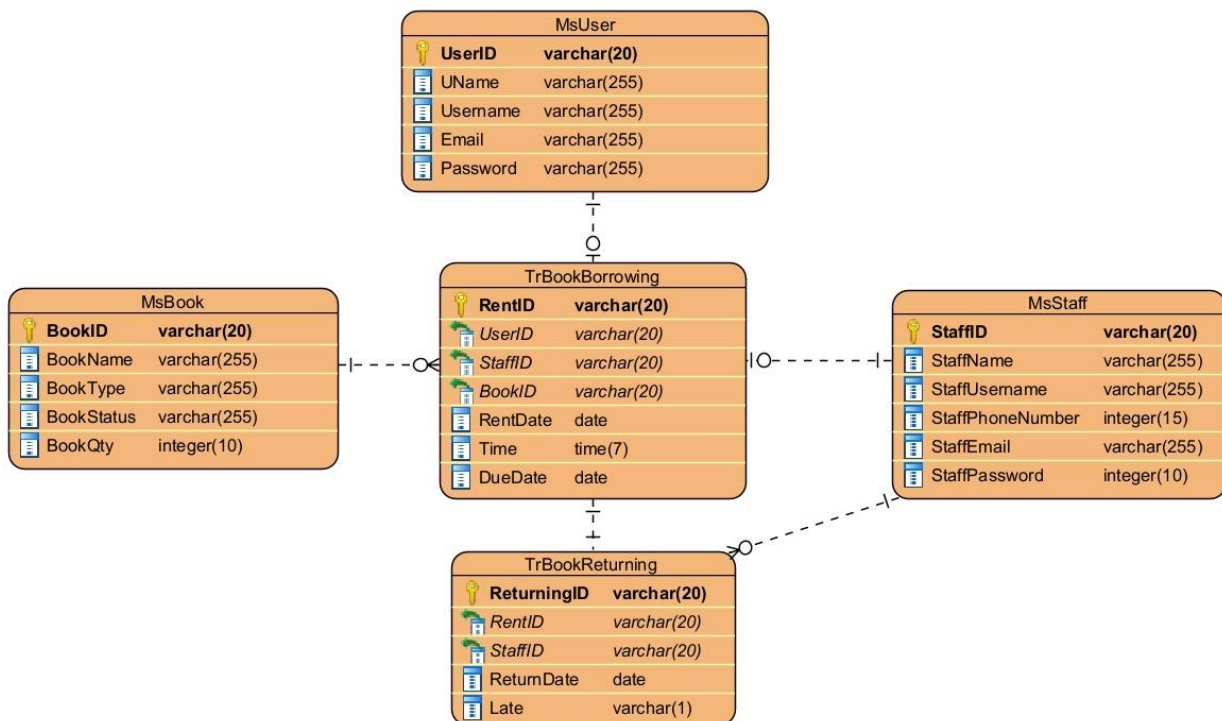
Percy Jackson #1: The Lightning Thief
Author: Rick Riordan
Ratings:

Rent Details
Rent Date : 25 September, 2023
Return Date : 25 October, 2023
Borrower : Jessica

Book RentBook Return

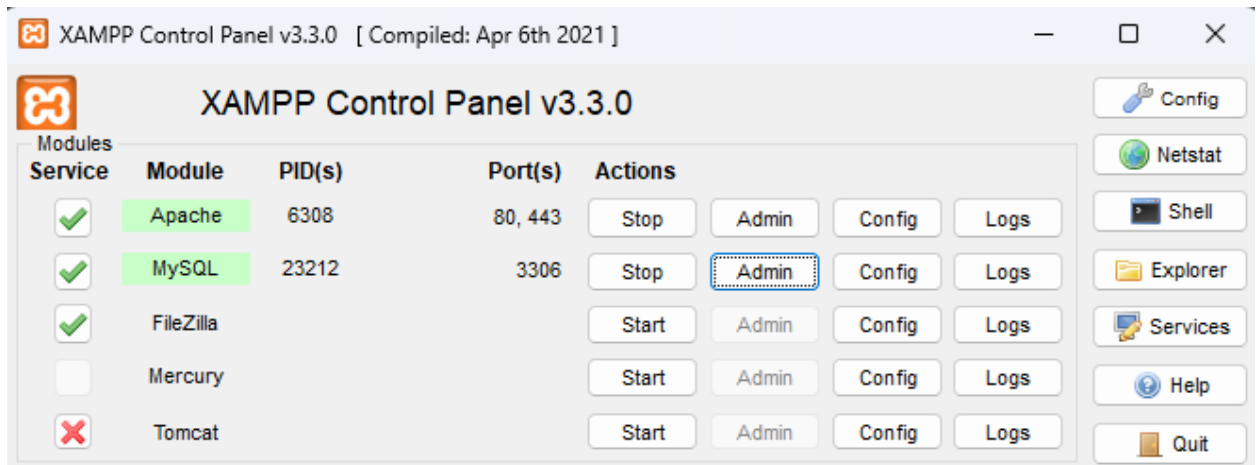
DB Design

1. ERD



- **MsUser - TrBookBorrowing**
Pada relasi tersebut, setiap User dapat tidak melakukan atau sekali melakukan BookBorrowing/Peminjaman Buku dan setiap BookBorrowing hanya bisa dilakukan oleh satu User.
- **MsBook - TrBookBorrowing**
Pada relasi tersebut, setiap Book dapat tidak memiliki atau memiliki banyak transaksi pada BookBorrowing/Peminjaman Buku dan setiap BookBorrowing harus ada satu Book.
- **MsStaff - TrBookBorrowing**
Pada relasi tersebut, setiap Staff dapat melakukan atau sekali melakukan BookBorrowing/Peminjaman Buku dan setiap BookBorrowing hanya bisa oleh satu Staff.
- **MsStaff - TrBookReturning**
Pada relasi tersebut, setiap Staff dapat tidak melayani atau banyak melayani BookReturning/Pengembalian Buku. dan setiap BookReturning hanya dapat dilayani oleh satu Staff.

2. XAMPP



Untuk Module XAMPP yang digunakan pada program Library Management System ini adalah Apache dan MySQL.

3. SQL QUERY

```
1 CREATE DATABASE Library;
2 USE Library;
3
4 CREATE TABLE `user` (
5     `id` INT AUTO_INCREMENT PRIMARY KEY,
6     `username` VARCHAR(255) NOT NULL,
7     `password` VARCHAR(255) NOT NULL,
8     `role` VARCHAR(50) NOT NULL
9 );
10
11 INSERT INTO `user` (`username`, `password`, `role`) VALUES
12     ('staff', 'staff', 'staff'),
13     ('customer', 'customer', 'customer');
14
15 CREATE TABLE `buku` (
16     `id` INT AUTO_INCREMENT PRIMARY KEY,
17     `judul` VARCHAR(255) NOT NULL,
18     `stok` INT NOT NULL
19 );
20
21 INSERT INTO `buku` (`judul`, `stok`) VALUES
22     ('Naruto', 10),
23     ('Boruto', 15),
24     ('Ayah', 20),
25     ('Uzumaki', 8),
26     ('Dilan 1990', 5),
27     ('Dilan: Dia Adalah Dilanku Tahun 1990', 12),
28     ('Dilan Bagian Kedua: Dia Adalah Dilanku Tahun 1991', 7),
29     ('Milea: Suara Dari Dilan', 18),
30     ('Ancika: Dia yang Bersamaku Tahun 1995', 9),
31     ('Tentang Kamu', 14),
32     ('Bumi', 25),
33     ('Hujan', 11),
34     ('Pulang-Pergi', 6),
35     ('Janji', 13),
36     ('Rembulan Tenggelam di Wajahmu', 16);
```

SQL Query yang kami buat tersebut membuat database bernama “Library” lalu menghasilkan 2 tabel yaitu ‘user’ dan ‘buku’ dimana attribut user adalah ‘id’, ‘username’, ‘password’, ‘role’. Kemudian untuk attribut buku adalah ‘id’, ‘judul’, ‘stok’. Dari kedua tabel tersebut telah di-insert beberapa data/values sesuai yang diperlukan.

Coding Explanation for DB Connectivity and GUI

1. DatabaseConnection.java

```
DatabaseConnection.java x
1 package main;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 public class DatabaseConnection {
8     private static final String DB_URL = "jdbc:mysql://localhost:3306/library";
9     private static final String DB_USER = "root";
10    private static final String DB_PASSWORD = "";
11
12    public Connection getConnection() throws SQLException {
13        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
14    }
15 }
```

Line 1: package main: mendefinisikan bahwa kelas ini termasuk dalam paket "main"

Line 3-5: Mengimpor kelas-kelas yang diperlukan dari paket java.sql untuk bekerja dengan JDBC (Java Database Connectivity), yaitu untuk mengelola koneksi database.

Line 8: Mendefinisikan URL database MySQL yang ingin diakses. Dalam hal ini, database berada di localhost (mesin yang sama dengan program) dan menggunakan port 3306 dengan nama database "library".

Line 9: Mendefinisikan username yang digunakan untuk login ke database.

Line 10: Mendefinisikan password yang digunakan untuk login ke database. (Dalam kasus ini, password dikosongkan.)

Method 'getConnection()'

Line 11: Mendefinisikan method getConnection() yang dapat melempar pengecualian SQLException

Line 12: Menggunakan DriverManager untuk mendapatkan koneksi ke database berdasarkan URL, username, dan password yang telah didefinisikan sebelumnya.

2. Book.java

```
Book.java x
1 package main;
2
3 import javafx.beans.property.SimpleIntegerProperty;
4 import javafx.beans.property.SimpleStringProperty;
5 import javafx.beans.property.IntegerProperty;
6 import javafx.beans.property.StringProperty;
7
8 public class Book {
9     private final StringProperty judul;
10    private final IntegerProperty stok;
11
12    public Book(String judul, int stok) {
13        this.judul = new SimpleStringProperty(judul);
14        this.stok = new SimpleIntegerProperty(stok);
15    }
16
17    public String getJudul() {
18        return judul.get();
19    }
20
21    public void setJudul(String judul) {
22        this.judul.set(judul);
23    }
24
25    public StringProperty judulProperty() {
26        return judul;
27    }
28
29    public int getStok() {
30        return stok.get();
31    }
32
33    public void setStok(int stok) {
34        this.stok.set(stok);
35    }
36
37    public IntegerProperty stokProperty() {
38        return stok;
39    }
40
41    public void pinjamBuku() {
42        if (stok.get() > 0) {
43            stok.set(stok.get() - 1);
44        }
45    }
46 }
```

Properti: `judul` dan `stok` adalah properti buku. `SimpleStringProperty` digunakan untuk properti judul, dan `SimpleIntegerProperty` digunakan untuk properti stok.

Konstruktor: Menerima judul dan stok awal sebagai parameter untuk membuat objek buku. Inisialisasi properti judul dan stok dengan nilai awal yang diberikan.

Getter dan Setter: Getter dan setter untuk mendapatkan dan mengatur nilai properti judul dan stok

Property Methods: `judulProperty()` dan `stokProperty()` mengembalikan properti judul dan stok sebagai objek `StringProperty` dan `IntegerProperty`

pinjamBuku() Method: Method ini mengurangi stok buku jika stok lebih dari 0. Digunakan untuk simulasi peminjaman buku.

3. LoginScene.java

```
1 package main;
2
3 import javafx.geometry.Insets;
4 import javafx.geometry.Pos;
5 import javafx.scene.Scene;
6 import javafx.scene.control.Button;
7 import javafx.scene.control.Label;
8 import javafx.scene.control.PasswordField;
9 import javafx.scene.control.Separator;
10 import javafx.scene.control.TextField;
11 import javafx.scene.image.Image;
12 import javafx.scene.image.ImageView;
13 import javafx.scene.layout.GridPane;
14 import javafx.scene.layout.HBox;
15 import javafx.scene.layout.VBox;
16 import javafx.stage.Stage;
17 import javafx.scene.control.Alert;
18 import javafx.scene.control.Alert.AlertType;
19
20 public class LoginScene extends Scene {
21     private UserService userService;
22     private UserDashboard userDashboard;
23
24     public LoginScene(UserService userService, UserDashboard userDashboard) {
25         super(new HBox(), 800, 400);
26         this.userService = userService;
27         this.userDashboard = userDashboard;
28
29         HBox loginHBox = (HBox) this.getRoot();
30         loginHBox.setSpacing(10);
31         loginHBox.setPadding(new Insets(10));
32
33         Image logoImage = new Image("file:///C:/Users/vanness/Downloads/Compressed/Project-LEC/assets/logoF.png");
34         ImageView logoImageView = new ImageView(logoImage);
35         logoImageView.setFitWidth(250); // Parlehan Logo
36         logoImageView.setPreserveRatio(true);
37
38         VBox leftVBox = new VBox();
39         leftVBox.setPrefWidth(400);
40
41         VBox rightVBox = new VBox();
42         rightVBox.setPrefWidth(400);
43         rightVBox.setStyle("-fx-background-color: #FFFFFF; -fx-font-family: 'Poppins'; -fx-alignment: center;"); // Set background color to #FFFFFF
44
45         Label titleLabel = new Label("Welcome to Librarie.");
46         titleLabel.setStyle("-fx-font-size: 24px; -fx-text-fill: #000000; -fx-font-weight: bold; -fx-font-family: 'Poppins';");
47
48         GridPane credentialsGridPane = new GridPane();
49
50         credentialsGridPane.add(passwordField, 1, 1);
51
52         VBox.setMargin(loginButton, new Insets(20, 0, 0, 0));
53
54         rightVBox.getChildren().addAll(titleLabel, new Separator(), credentialsGridPane, loginButton);
55
56         loginHBox.getChildren().addAll(leftVBox, rightVBox);
57
58         loginButton.setOnAction(e -> {
59             String username = usernameField.getText();
60             String password = passwordField.getText();
61
62             if (userService.validateLogin(username, password)) {
63                 userDashboard.setLoggedInUsername(username);
64                 Stage stage = new Stage();
65                 stage.setTitle("User Dashboard");
66                 stage.setScene(new Scene(new UserDashboard(userService, userDashboard)));
67                 stage.show();
68             }
69         });
70     }
71 }
```

Konstruktor:

Menerima objek UserService dan UserDashboard untuk mengelola autentikasi pengguna dan navigasi antar layar.

Membangun tampilan login dengan menggunakan elemen-elemen JavaFX seperti Label, TextField, dan Button.

Element UI:

Menambahkan elemen-elemen seperti gambar logo, label judul, form login, dan tombol login ke dalam layout JavaFX.

Event Handler loginButton.setOnAction:

Ketika tombol login ditekan, mengambil input username dan password.

Melakukan validasi login menggunakan UserService.

Jika login berhasil, menetapkan username yang masuk ke dalam userDashboard dan menavigasi ke dashboard sesuai peran pengguna.

Jika login gagal, menampilkan alert kesalahan menggunakan metode showErrorAlert.

Metode showErrorAlert:

Menampilkan Alert dengan tipe kesalahan, judul, dan pesan yang diberikan.

4. UserServices.java

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

Baris-baris di atas merupakan import statement yang diperlukan untuk mengakses kelas-kelas yang terdapat di paket 'java.sql'. Paket ini menyediakan API untuk mengakses dan memanipulasi data dalam database.

```
public class UserService {
    private DatabaseConnection dbConnection;
```

Kelas 'UserService' adalah kelas utama yang menyediakan fungsionalitas terkait pengguna. Ada atribut 'dbConnection' yang bertipe 'DatabaseConnection'. Ini adalah objek yang bertanggung jawab untuk koneksi ke database.

```
    public UserService() {
        dbConnection = new DatabaseConnection();
    }
```

Konstruktor kelas 'UserService' membuat objek 'DatabaseConnection' saat objek UserService dibuat. Ini dilakukan untuk memastikan bahwa kita memiliki koneksi database yang siap digunakan.

```
    public boolean validateLogin(String username, String password) {
        try (Connection connection = dbConnection.getConnection()) {
            String query = "SELECT * FROM user WHERE username = ? AND password = ?";
            PreparedStatement preparedStatement = connection.prepareStatement(query);
            preparedStatement.setString(1, username);
            preparedStatement.setString(2, password);

            ResultSet resultSet = preparedStatement.executeQuery();
            boolean isValid = resultSet.next();

            resultSet.close();
            preparedStatement.close();

            return isValid;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        }
    }
```

Metode 'validateLogin' digunakan untuk memvalidasi login pengguna. Ini memeriksa apakah ada baris di tabel pengguna (user) dengan nama pengguna (username) dan kata sandi (password) yang sesuai.

- `Connection connection = dbConnection.getConnection()`

Mendapatkan objek koneksi dari `DatabaseConnection`.

- `String query = "SELECT * FROM user WHERE username = ? AND password = ?";`

Membuat string query SQL untuk mendapatkan pengguna berdasarkan nama pengguna dan kata sandi.

- `PreparedStatement preparedStatement = connection.prepareStatement(query);`

Mempersiapkan pernyataan SQL dengan memasukkan parameter (nama pengguna dan kata sandi) ke dalam query.

- `preparedStatement.setString(1, username);` dan `preparedStatement.setString(2, password);`

Mengatur nilai parameter pada `preparedStatement`.

- `ResultSet resultSet = preparedStatement.executeQuery();`

Mengeksekusi query dan mendapatkan hasil dalam bentuk `ResultSet`.

- `boolean isValid = resultSet.next();`

Memeriksa apakah ada hasil dalam `ResultSet`.

- `resultSet.close();` dan `preparedStatement.close();`

Menutup `ResultSet` dan `PreparedStatement` untuk membebaskan sumber daya.

- `return isValid;`

Mengembalikan nilai 'true' jika login valid, 'false' jika tidak. Jika terjadi kesalahan selama eksekusi query, akan mencetak stack trace kesalahan dan mengembalikan false.

```

public String getUserRole(String username) {
    try (Connection connection = dbConnection.getConnection()) {
        String query = "SELECT role FROM user WHERE username = ?";
        PreparedStatement preparedStatement = connection.prepareStatement(query);
        preparedStatement.setString(1, username);

        ResultSet resultSet = preparedStatement.executeQuery();

        if (resultSet.next()) {
            String userRole = resultSet.getString("role");
            resultSet.close();
            preparedStatement.close();
            return userRole;
        }

        resultSet.close();
        preparedStatement.close();

    } catch (SQLException e) {
        e.printStackTrace();
    }
    return null;
}

```

Metode `getUserRole` digunakan untuk mendapatkan peran pengguna berdasarkan nama pengguna.

- `Connection connection = dbConnection.getConnection()`
untuk mendapatkan objek koneksi.
- Query SQL diatur untuk mendapatkan peran dari tabel pengguna berdasarkan nama pengguna.
- Parameter (nama pengguna) diatur pada `preparedStatement`.
- Eksekusi query dan mendapatkan hasil dalam bentuk 'ResultSet'.
- Mengecek apakah ada hasil dengan 'if (resultSet.next()).'
- Jika ada hasil, mendapatkan nilai peran dari kolom "role" dalam 'ResultSet'.
- Menutup `ResultSet` dan `PreparedStatement` untuk membebaskan sumber daya.
- Mengembalikan nilai peran pengguna atau null jika tidak ada hasil atau terjadi kesalahan.

5. UserDashboard.java

```
package main;

import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Priority;
import javafx.scene.layout.Region;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.stage.Stage;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.cell.PropertyValueFactory;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class UserDashboard extends Application {

    private String loggedInUsername;
    private static final String DB_URL = "jdbc:mysql://localhost:3306/library";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "";
    private TableView<Book> bookTable;

    @Override
    public void start(Stage primaryStage) {
        primaryStage.setTitle("User Dashboard");

        VBox dashboardVBox = new VBox();
        dashboardVBox.setSpacing(10);
        dashboardVBox.setPadding(new Insets(10));
        dashboardVBox.setStyle("-fx-background-color: #f0f0f0; -fx-font-family: 'Poppins'");

        HBox menuBarContainer = new HBox();
        menuBarContainer.setSpacing(10);
        menuBarContainer.setPadding(new Insets(10));

        Label logoutLabel = new Label("Log Out");
        logoutLabel.setStyle("-fx-text-fill: white; -fx-font-family: 'Poppins'");
        logoutLabel.setOnMouseClicked(e -> logout(primaryStage));

        MenuBar menuBar = new MenuBar();
        menuBar.setStyle("-fx-background-color: #F47061; -fx-text-fill: white; -fx-font-weight: bold; -fx-font-family: 'Poppins'");

        Menu navigationMenu = new Menu();
        navigationMenu.setStyle("-fx-background-color: #F47061");
        navigationMenu.setGraphic(logoutLabel);

        MenuItem logoutMenuItem = new MenuItem();
        logoutMenuItem.setOnAction(e -> logout(primaryStage));

        navigationMenu.getItems().add(logoutMenuItem);

        menuBar.getMenus().add(navigationMenu);

        dashboardVBox.getChildren().add(menuBar);

        Region spacer = new Region();
        menuBarContainer.getChildren().addAll(spacer, menuBar);
        dashboardVBox.getChildren().add(menuBarContainer);

        Label welcomeLabel = new Label("Selamat datang, " + loggedInUsername);
        welcomeLabel.setStyle("-fx-font-size: 20px; -fx-text-fill: #333; -fx-font-family: 'Poppins'");

        TextField searchField = new TextField();
        Button searchButton = new Button("Cari Buku");
    }
}
```

```

Button actionButton;
if (isStaffUser()) {
    actionButton = new Button("Tambah Buku");
    actionButton.setStyle("-fx-background-color: #4CAF50; -fx-text-fill: white; -fx-font-family: 'Poppins'");

    actionButton.setOnAction(e -> {
        TextInputDialog dialog = new TextInputDialog();
        dialog.setTitle("Tambah Buku");
        dialog.setHeaderText("Masukkan detail buku");
        dialog.setContentText("Judul Buku:");

        TextField judulField = new TextField();
        TextField stokField = new TextField();

        VBox content = new VBox(8, new Label("Judul Buku:"), judulField, new Label("Stok Buku:"), stokField);

        dialog.getDialogPane().setContent(content);
        Optional<String> result = dialog.showAndWait();
        if (result.isPresent()) {
            String judulBuku = judulField.getText().trim();
            String stokInput = stokField.getText().trim();

            if (!judulBuku.isEmpty() && !stokInput.isEmpty()) {
                try {
                    int stok = Integer.parseInt(stokInput);

                    if (tambahBukuKeDatabase(judulBuku, stok)) {
                        showInformationDialog("Sukses", "Buku berhasil ditambahkan ke dalam sistem.");
                        refreshListView();
                    } else {
                        showErrorDialog("Kesalahan", "Penambahan buku gagal. Silakan coba lagi.");
                    }
                } catch (NumberFormatException ex) {
                    showErrorDialog("Kesalahan", "Masukkan stok buku dengan nilai numerik.");
                }
            } else {
                showErrorDialog("Kesalahan", "Judul buku dan stok buku harus diisi.");
            }
        }
    });
}

} else {
    actionButton = new Button("Pinjam Buku");
    actionButton.setStyle("-fx-background-color: #4CAF50; -fx-text-fill: white; -fx-font-family: 'Poppins'");
    actionButton.setDisable(true);
    actionButton.setOnAction(e -> {
        Book selectedBook = bookTable.getSelectionModel().getSelectedItem();
        if (selectedBook != null) {
            if (hapusBukuDariDatabase(selectedBook)) {
                String transactionID = generateTransactionID();
                Alert alert = new Alert(Alert.AlertType.INFORMATION);
                alert.setTitle("Sukses");
                alert.setHeaderText(null);
                alert.setContentText("Transaksi " + transactionID + " berhasil. Buku berhasil dipinjam.");
                alert.showAndWait();
            }
        }
    });
}

```

```

bookTable.getSelectionModel().selectedItemProperty().addListener((obs, oldSelection, newSelection) -> {
    if (newSelection != null) {
        actionButton.setDisable(false);
    } else {
        actionButton.setDisable(true);
    }
});

HBox.setHgrow(actionButton, Priority.ALWAYS);
HBox.setMargin(actionButton, new Insets(0, 10, 0, 0));
actionButton.setMaxWidth(Double.MAX_VALUE);

buttonBox.getChildren().add(actionButton);

dashboardVBox.getChildren().addAll(welcomeLabel, searchBox, bookTable, buttonBox);

searchButton.setOnAction(e -> {
    String searchTerm = searchField.getText();
    searchBooks(searchTerm);
});

loadBooks();

Scene dashboardScene = new Scene(dashboardVBox, 800, 400);

primaryStage.setScene(dashboardScene);
primaryStage.show();
}

private void showInformationDialog(String title, String contentText) {
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setTitle(title);
    alert.setHeaderText(null);
    alert.setContentText(contentText);
    alert.showAndWait();
}

private void showErrorDialog(String title, String contentText) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle(title);
    alert.setHeaderText(null);
    alert.setContentText(contentText);
    alert.showAndWait();
}

private String generateTransactionID() {
    int randomDigits = (int) (Math.random() * 1000);
    return String.format("TR%03d", randomDigits);
}

```

```

private boolean pinjamBuku(Book selectedBook) {
    try {
        Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

        if (selectedBook.getStok() > 0) {
            String query = "UPDATE buku SET stok = stok - 1 WHERE judul = ?";
            try (PreparedStatement preparedStatement = connection.prepareStatement(query)) {
                preparedStatement.setString(1, selectedBook.getJudul());
                int rowsAffected = preparedStatement.executeUpdate();

                preparedStatement.close();
                connection.close();

                return rowsAffected > 0;
            }
        } else {
            connection.close();
            return false;
        }
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

```

```

private boolean hapusBukuDariDatabase(Book selectedBook) {
    try {
        Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

        if (selectedBook.getStok() > 0) {
            String query = "UPDATE buku SET stok = stok - 1 WHERE judul = ?";
            try (PreparedStatement preparedStatement = connection.prepareStatement(query)) {
                preparedStatement.setString(1, selectedBook.getJudul());
                int rowsAffected = preparedStatement.executeUpdate();

                preparedStatement.close();
                connection.close();

                return rowsAffected > 0;
            }
        } else {
            connection.close();
            return false;
        }
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

```

```

private boolean isStaffUser() {
    if (loggedInUsername == null) {
        return false;
    }

    try {
        Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

```

```

private List<Book> performSearchInDatabase(String searchTerm) {
    List<Book> searchResults = new ArrayList<>();

    try {
        Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

        String query = "SELECT * FROM buku WHERE judul LIKE ?";
        System.out.println("SQL Query: " + query);

        try (PreparedStatement preparedStatement = connection.prepareStatement(query)) {
            preparedStatement.setString(1, "%" + searchTerm + "%");

            try (ResultSet resultSet = preparedStatement.executeQuery()) {
                while (resultSet.next()) {
                    String judul = resultSet.getString("judul");
                    int stok = resultSet.getInt("stok");
                    searchResults.add(new Book(judul, stok));
                }
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }

    return searchResults;
}

public void setLoggedInUsername(String username) {
    loggedInUsername = username;
}

private void refreshListView() {
    List<Book> allBooks = getAllBooksFromDatabase();
    bookTable.getItems().setAll(allBooks);
}

private boolean tambahBukuKeDatabase(String judulBuku, int stok) {
    if (judulBuku == null || judulBuku.trim().isEmpty()) {
        System.out.println("Judul buku tidak boleh kosong.");
        return false;
    }
    try {
        Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

```

```

private void loadBooks() {
    bookTable.getItems().clear();
    List<Book> allBooks = getAllBooksFromDatabase();
    bookTable.getItems().addAll(allBooks);
}

private List<Book> getAllBooksFromDatabase() {
    List<Book> allBooks = new ArrayList<>();

    try {
        Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

        String query = "SELECT judul, stok FROM buku";
        try (PreparedStatement preparedStatement = connection.prepareStatement(query)) {
            try (ResultSet resultSet = preparedStatement.executeQuery()) {
                while (resultSet.next()) {
                    String judul = resultSet.getString("judul");
                    int stok = resultSet.getInt("stok");
                    allBooks.add(new Book(judul, stok));
                }
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }

    return allBooks;
}

```

Penjelasan:

- Deklarasi Variabel dan Konstanta
 - loggedInUsername: Menyimpan nama pengguna yang sedang masuk.
 - DB_URL, DB_USER, DB_PASSWORD: Konstanta untuk informasi koneksi ke database.
 - bookTable: Tabel JavaFX untuk menampilkan data buku.
- Metode start(Stage primaryStage)

Adalah metode utama yang mendefinisikan antarmuka pengguna.

A. Pengaturan Awal

- 'VBox dashboardVBox': Kontainer utama dengan tata letak vertikal.
- 'HBox menuBarContainer': Kontainer untuk menampung menu bar.
- 'Label logoutLabel': Label untuk tindakan logout.
- 'MenuBar menuBar': Menu bar untuk navigasi.

B. Komponen Grafis

- 'Label welcomeLabel': Label untuk menyapa pengguna.
- 'TextField' 'searchField' dan 'Button searchButton': Input pencarian buku.
- 'TableView<Book> bookTable': Tabel untuk menampilkan daftar buku.
- Kolom tabel: 'titleColumn' (judul buku) dan 'stockColumn' (stok buku).
- 'HBox buttonBox': Kontainer untuk tombol aksi (Tambah Buku atau Pinjam Buku).

C. Tombol Aksi

- Tombol aksi ('actionButton') menyesuaikan peran pengguna.
- Menggunakan 'TextInputDialog' untuk mendapatkan detail buku baru.
- Aksi tombol mengacu pada metode yang sesuai ('tambahBukuKeDatabase' atau 'pinjamBuku').

D. Listener dan Handler

- 'bookTable.getSelectionModel().selectedItemProperty()': Listener untuk mengaktifkan tombol aksi saat item dipilih di tabel.
- 'searchButton.setOnAction(e -> searchBooks(searchTerm))': Handler untuk pencarian buku.

E. Menampilkan Dashboard

- 'Scene dashboardScene': Membuat antarmuka pengguna dengan ukuran 800x400 piksel.
- 'primaryStage.setScene(dashboardScene)': Menetapkan antarmuka pengguna ke panggung utama.
- 'primaryStage.show()': Menampilkan panggung utama.

● Metode Utilitas

- 'showInformationDialog(String title, String contentText)': Menampilkan dialog informasi.
- 'showErrorDialog(String title, String contentText)': Menampilkan dialog kesalahan.
- 'generateTransactionID()': Menghasilkan ID transaksi acak.
- 'pinjamBuku(Book selectedBook)': Metode untuk melakukan peminjaman buku.
- 'hapusBukuDariDatabase(Book selectedBook)': Metode untuk menghapus buku dari database.
- 'isStaffUser()': Menentukan apakah pengguna yang masuk memiliki peran "staff".

- 'searchBooks(String searchTerm)': Mencari buku berdasarkan kata kunci.
- 'performSearchInDatabase(String searchTerm)': Melakukan pencarian buku di database.
- 'setLoggedInUsername(String username)': Menetapkan nama pengguna yang masuk.
- 'logout(Stage primaryStage)': Metode untuk logout dan membuka kembali layar login.
- 'refreshListView()': Memperbarui tampilan daftar buku.
- 'tambahBukuKeDatabase(String judulBuku, int stok)': Menambahkan buku baru ke database.
- 'loadBooks()': Memuat daftar buku dari database.
- 'getAllBooksFromDatabase()': Mengambil semua buku dari database.

6. Main.java

Kode tersebut merupakan kelas utama (Main) dari aplikasi perpustakaan (Library Management System) yang menggunakan JavaFX.

```
import javafx.application.Application;
import javafx.stage.Stage;
```

- Mengimpor kelas-kelas yang diperlukan dari pustaka/library JavaFX, termasuk Application dan Stage.

```
public class Main extends Application {
```

- Mendeklarasikan kelas Main yang mengimplementasikan antarmuka (interface) Application dari JavaFX.

```
private UserDashboard userDashboard;
private UserService userService;
```

- Mendeklarasikan dua variabel instance ('userDashboard' dan 'userService') yang akan digunakan dalam kelas Main.

```
public Main() {
    userDashboard = new UserDashboard();
    userService = new UserService();
}
```

- Konstruktor kelas Main yang menginisialisasi objek userDashboard dan userService saat kelas Main dibuat.

```
public static void main(String[] args) {
    launch(args);
}
```

- Metode utama (main) yang memulai eksekusi program. Metode ini memanggil launch(args) untuk memulai aplikasi JavaFX.

```
@Override
public void start(Stage primaryStage) {
    primaryStage.setTitle("Library Management System");
    primaryStage.setScene(new LoginScene(userService, userDashboard));
    primaryStage.show();
}
```

- Override dari metode start yang ada di kelas Application. Metode ini akan dijalankan ketika aplikasi dimulai.
- 'primaryStage.setTitle("Library Management System")': Menetapkan judul pada jendela aplikasi.

- 'primaryStage.setScene(new LoginScene(userService, userDashboard))': Menetapkan antarmuka pengguna awal menggunakan LoginScene dengan menyediakan objek 'userService' dan 'userDashboard'.
- 'primaryStage.show()': Menampilkan jendela aplikasi.

7. Module-info.java

```
module YourModuleName {
    requires javafx.controls;
    requires javafx.fxml;
    requires java.sql;
    requires javafx.graphics;
    exports main;
    opens main to javafx.base;
}
```

- module YourModuleName {: Mendeklarasikan modul Java baru dengan nama "YourModuleName."
- requires javafx.controls;: Menyatakan bahwa modul ini membutuhkan modul JavaFX controls. Ini berarti kelas-kelas dalam modul ini bergantung pada kelas-kelas dari modul javafx.controls.
- requires javafx.fxml;: Menyatakan bahwa modul ini membutuhkan modul JavaFX FXML. Ini berarti kelas-kelas dalam modul ini bergantung pada kelas-kelas dari modul javafx.fxml.
- requires java.sql;: Menyatakan bahwa modul ini membutuhkan modul Java SQL. Ini berarti kelas-kelas dalam modul ini bergantung pada kelas-kelas dari modul java.sql.
- requires javafx.graphics;: Menyatakan bahwa modul ini membutuhkan modul JavaFX graphics. Ini berarti kelas-kelas dalam modul ini bergantung pada kelas-kelas dari modul javafx.graphics.
- exports main;: Membuat paket main (atau modul) dapat diakses oleh modul lain. Kelas-kelas dalam paket main dapat digunakan oleh kelas-kelas dalam modul lain yang membutuhkan modul ini.
- opens main to javafx.base;: Membuka paket main untuk modul javafx.base, memungkinkan refleksi untuk mengakses bagian dalam kelas-kelas dalam paket main. Ini sering diperlukan saat menggunakan JavaFX dengan berkas FXML yang mungkin membutuhkan akses refleksi.

Question and Answer

- Apa yang membuat kalian yakin apa yang kalian buat bisa lebih baik dari app library lain?
 - Sistem pada aplikasi kami mudah digunakan baik itu oleh pengguna yang memahami teknologi secara luas maupun pengguna baru. Sistem kami juga menyediakan tampilan yang menarik untuk dilihat oleh pengguna.
- Kenapa admin input data menggunakan judul buku, bukan id buku?
 - Karena pada sistem kami id buku tercipta setelah admin memasukkan judul buku terlebih dahulu.

Chapter III - Conclusion

Setelah menjalani proses pengembangan hingga presentasi, kami dapat menyatakan bahwa proyek ini sukses. Kami berhasil mencapai objektif yang kami tetapkan yaitu berhasil mengembangkan sebuah sistem administrasi yang bertujuan untuk membantu perpustakaan. Secara fungsi, sistem kami berjalan dengan efisien dan sesuai dengan tujuannya, seluruh fitur mulai dari login, pencarian buku, penambahan buku, dan peminjaman buku berjalan dengan baik dan tanpa hambatan. Akses informasi yang ditunjukkan kepada pengguna sudah akurat dan memiliki kecepatan feedback yang sangat cepat. Seluruh anggota kelompok menjalankan tugasnya masing-masing sesuai dengan pembagian pekerjaan meskipun terdapat beberapa hambatan, para anggota berhasil menyesuaikan dan menjalankan tugas dengan baik. Proyek ini diselesaikan tepat waktu, berdasarkan timeline yang kami setuju dan tetapkan maka proyek ini berhasil memenuhi tuntutan tersebut.

Setelah melakukan presentasi, kami mendapatkan saran-saran yang membangun dan juga kritik yang memotivasi kami dalam membenahi sistem yang kami rancang. Sesuai dengan timeline yang ditetapkan maka kami menjalankan beberapa perbaikan agar dapat memenuhi saran serta tambahan yang diberikan. Untuk beberapa hal yang dapat kita tingkatkan atau perbaiki antara lain adalah perincian pemberian informasi mengenai buku-buku yang tersedia dalam koleksi. Secara garis besar, kami merasa dengan waktu yang diberikan dan juga sumber daya yang dimiliki bahwa proyek ini berjalan dengan baik dan melihat adanya potensi untuk mengembangkan sistem yang kami rancang menjadi lebih baik.

Reference

- vpadmin. (2023, March 20). *Elaborating Use Cases with Activity Diagrams: Visualizing Scenarios for Normal, Alternative, and Exception Paths - Visual Paradigm Guides*. Visual Paradigm Guides. <https://guides.visual-paradigm.com/elaborating-use-cases-with-activity-diagrams-visualizing-scenarios-for-normal-alternative-and-exception-paths/>
- *Activity Diagram - Activity Diagram Symbols, Examples, and More*. (2022). Smartdraw.com. <https://www.smartdraw.com/activity-diagram/>
- *UML Activity Diagram Tutorial*. (2023). Lucidchart. <https://www.lucidchart.com/pages/uml-activity-diagram>
- *JavaFX Tutorial - javatpoint*. (2014). Wwww.javatpoint.com. <https://www.javatpoint.com/javafx-tutorial>
- *JavaFX Tutorial*. (2023). Tutorialspoint.com. <https://www.tutorialspoint.com/javafx/index.htm>
- *How to connect to database in Java | Java Database Connectivity - javatpoint*. (2021). Wwww.javatpoint.com. <https://www.javatpoint.com/steps-to-connect-to-the-database-in-java>
- *What is Entity Relationship Diagram (ERD)?* (2023). Visual-Paradigm.com. <https://www.visual-paradigm.com/guide/data-modeling/what-is-entity-relationship-diagram/>
- Setiawan, R. (2021, August 24). *Memahami ERD, Model Data, dan Komponennya - Dicoding Blog*. Dicoding Blog. <https://www.dicoding.com/blog/memahami-erd/>
- *What is an Entity Relationship Diagram (ERD)?* (2023). Lucidchart. <https://www.lucidchart.com/pages/er-diagrams>

- Neso Academy. (2021). Basic Concepts of Entity-Relationship Model [YouTube Video]. In *You Tube*. <https://www.youtube.com/watch?v=wOD02sezmx8>
- ProgrammingKnowledge. (2015). JavaFx Tutorial For Beginners 1 - Introduction To JavaFx [YouTube Video]. In *You Tube*. https://www.youtube.com/watch?v=9YrmON6nlEw&list=PLS1QulWo1RIaUGP446_pWLgTZPiFizEMq
- Chaudhary, A. (2022). Java connectivity with MySQL database in Eclipse | Download MySQL connector driver [YouTube Video]. In *You Tube*. <https://www.youtube.com/watch?v=bmv5SLrEQ-M>
- Saad, P. (2017). JavaFX - Show Image [YouTube Video]. In *You Tube*. <https://www.youtube.com/watch?v=kR-jnEVzzrA>
- Barde, V. (2022). JDBC To MySql Connection - XAMPP SERVER(Eclipse 2022) [YouTube Video]. In *You Tube*. <https://www.youtube.com/watch?v=NuuuewLqBtE>
- [Dribble-Website-Mockup.png](#)
- [Library-management-system-project-using-python-1024x488.jpg](#)
- [Template-Library-Feature.jpg](#)
- [Website-mockups.png](#)
- [MicrosoftTeams-image \(1\).png](#)
- [MicrosoftTeams-image.png](#)